

SpikeCV: open a continuous computer vision era

Yajing ZHENG¹, Jiyuan ZHANG¹, Rui ZHAO¹, Jianhao DING¹, Shiyan CHEN¹,
Weijian WU³, Ruiqin XIONG¹, Zhaofei YU^{1,2} & Tiejun HUANG^{1,2*}

¹State Key Laboratory for Multimedia Information Processing, School of Computer Science,
Peking University, Beijing 100871, China

²Institute for Artificial Intelligence, Peking University, Beijing 100871, China

³SpikeSee Technology Co., Ltd., Beijing 100084, China

Received 19 July 2023/Revised 23 October 2024/Accepted 8 May 2025/Published online 14 January 2026

Abstract SpikeCV is a new open-source computer vision platform designed for the spike camera, a neuromorphic visual sensor that has seen rapid development in recent years. Unlike traditional cameras, the spike camera operates without a global shutter; instead, each pixel accumulates light intensity and asynchronously fires spikes when a preset threshold is reached. These binary spikes can be generated at frequencies up to 40000 Hz, offering a new mode of visual representation with high spatiotemporal resolution, capturing continuous visual information from the environment. Taking advantage of the spike camera's low latency and high dynamic range, many spike-based algorithms have made significant progress, such as high-quality imaging and ultra-high-speed target detection. To foster a collaborative ecosystem for spike vision, SpikeCV provides ultra-high-speed scene datasets, hardware interfaces, and a comprehensive module library. It emphasizes data encapsulation, standardization of dataset interfaces, modularity for vision tasks, and support for real-time applications in challenging scenarios. Unlike existing vision frameworks, SpikeCV is specifically optimized for spike data and real-time processing, addressing the unique requirements of spike-based systems. With the advent of the open-source Python ecosystem, modules of SpikeCV can be used as a Python library to fulfill most of the numerical analysis needs of researchers. We demonstrate the efficiency of the SpikeCV on offline inference and real-time applications. The project repository address and website are <https://openi.pcl.ac.cn/Cordium/SpikeCV> and <https://spikecv.github.io>.

Keywords spike camera, datasets, spike-based algorithms, spike vision, open-source Python ecosystem

Citation Zheng Y J, Zhang J Y, Zhao R, et al. SpikeCV: open a continuous computer vision era. *Sci China Inf Sci*, 2026, 69(3): 132106, <https://doi.org/10.1007/s11432-023-4565-6>

1 Introduction

Traditional digital cameras capture visual information as a continuous sequence of frames, but there is a fixed exposure time interval between each frame. This results in the loss of visual changes that occur during these intervals. Visual changes during the interval are thus lost. For changing scenes, recording continuous vision should pay more attention to the light signal itself. As noted by von Neumann in *The Computers and the Brain* [1], discrete nerve spikes can express continuous analog values, such as light and sound. The spike camera [2, 3] eliminates the traditional concept of a shutter, as each pixel fires a spike whenever the accumulated photon intensity reaches a preset threshold. In the spike stream, regions with higher scene brightness generate spikes more frequently, reflecting the continuous variation in light intensity.

Spike camera continuously capturing visual data offers significant advantages in challenging scenarios, such as analyzing high-speed molecular or object motion [4], monitoring industrial machining processes [5, 6], and enabling rapid obstacle avoidance in drones and autonomous vehicles [7, 8]. In recent years, taking advantage of the low latency and high dynamic range (HDR) of the spike camera, many spike-based algorithms have emerged, including high-quality image reconstruction [9–13], optical flow [14, 15], depth estimation [16, 17], and ultra-high-speed target detection and tracking [3, 18], and HDR imaging [19–21]. In addition to applying spike cameras to vision tasks, there are studies dedicated to coding and simulators for spike cameras [14, 15].

Unlike traditional vision systems that rely on frame sequences, spike cameras output a continuous stream of bits representing light changes at extremely high speeds. However, almost all tools in computer vision serve image sequences, such as OpenCV [22]. As the Chinese proverb goes, ‘To do a good job, one needs the right tools’. While

* Corresponding author (email: tjhuang@pku.edu.cn)

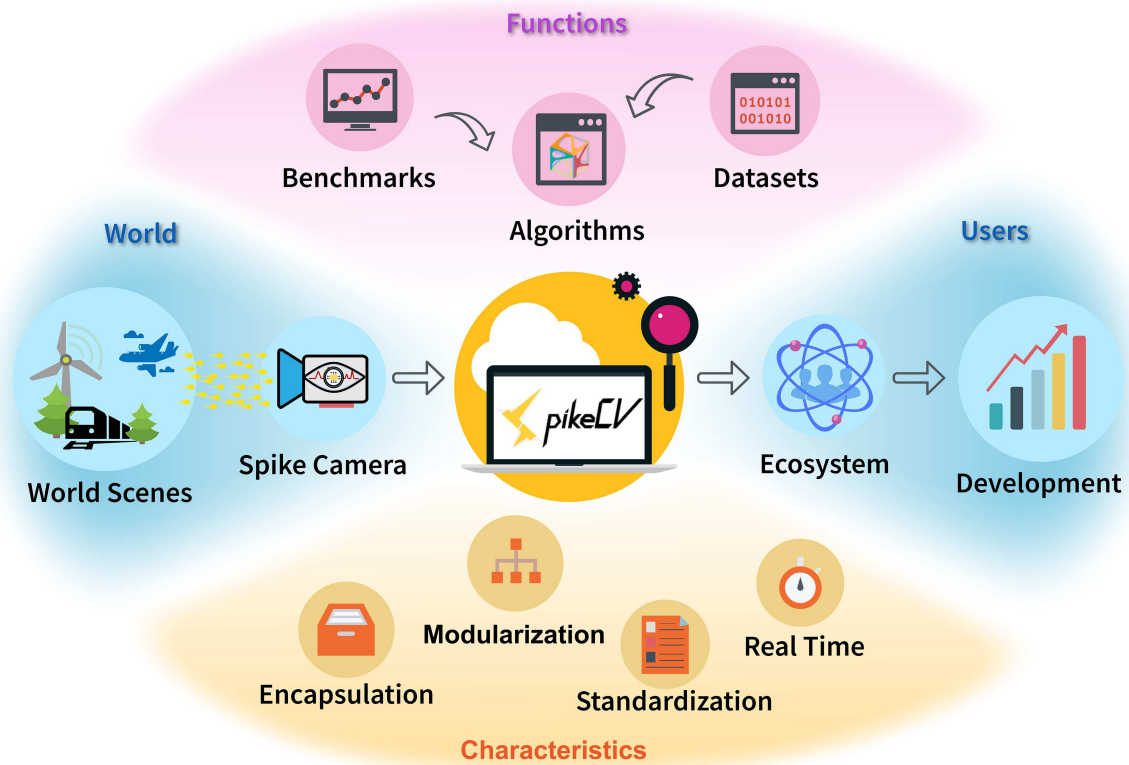


Figure 1 (Color online) Core design principles of SpikeCV: SpikeCV enables the capture of continuous visual information from the world using spike cameras and supports the growth of the spike vision ecosystem.

spike cameras and spike-based algorithms have made significant progress, the steep learning curve for beginners in spike vision remains a barrier. Therefore, to better promote the advancement of machine vision using continuous vision based on spikes, we build a spike-based open-source platform for spike cameras, SpikeCV.

The key challenge in creating SpikeCV is standardizing datasets across different vision tasks and providing a unified data interface for both offline files and real-time spike streams. Integrating datasets and perception hardware is a critical issue in computer vision, often involving hardware data caching, decoding, and the creation of appropriate file directory structures [23–25]. These elements ensure that various types of spike data can be processed seamlessly through a single interface. Unlike existing frameworks that primarily focus on image processing and algorithms [22, 26], SpikeCV offers tools specifically for spike processing, hardware interfaces for spike cameras, and standardized datasets for spike-based vision tasks.

SpikeCV is primarily written in Python, making it accessible to users familiar with numerical analysis toolkits like NumPy [27], SciPy [28], and PyTorch [29]. C++ is used for real-time interaction with hardware. An intuitive illustration of the design principles of SpikeCV is shown in Figure 1. SpikeCV is a bridge between spike cameras and users. By providing different functional modules and easy-to-use packaging interfaces, standardized data, real-time processing pipelines, and modular spike-based tools, it helps to form a spike vision community and promote the development of continuous vision scene applications. In this paper, we introduce the design principles, working mechanism, and usage of SpikeCV in detail. In the experiments, we demonstrate the efficiency of SpikeCV by achieving multiple vision tasks in the same scene offline and in real-time, respectively. Our contributions can be summarized as follows.

- We provide the first open-source platform for spike cameras, encapsulate datasets, device interface, spike loader, and spike-based algorithms, to make it easy to build spike vision applications.
- We standardize the datasets of various vision algorithms and provide a unified data interface for different visual tasks, offline files, and real-time data from spike cameras.
- We realize the real-time pipeline for hardware reading of various spike cameras by providing a hardware-friendly C++ and user-friendly Python interface, which achieves the synchronization between reading and processing of spike data.

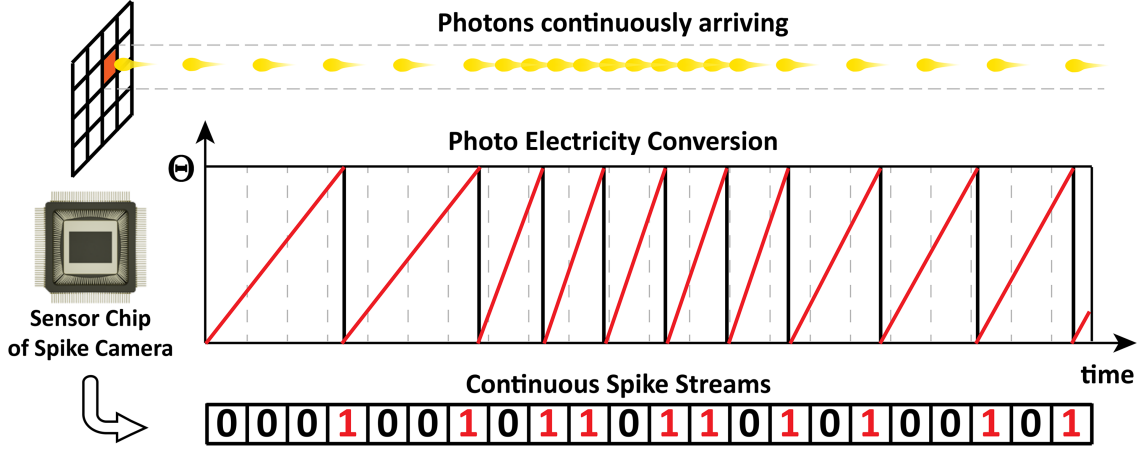


Figure 2 (Color online) Mechanism of the spike camera.

2 Background

2.1 Spike camera

The sensor of a spike camera is arranged by $H \times W$ pixel units that continuously receive photons and fire spikes [2]. As illustrated in Figure 2, the instant light intensity $L_{i,j}(t)$ at position (i, j) will be converted to an electrical signal to increase voltage $V_{i,j}(t)$ at time t . The voltage $V_{i,j}(t)$ will be reset to 0 when it reaches a threshold Θ , at which a spike will be fired. The process can be formulated as

$$V_{x,y}^+(t) = \begin{cases} V_{i,j}^-(t) + L_{i,j}(t), & \text{if } V_{i,j}^-(t) < \Theta, \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

The back-end synchronous circuit always poll the $V(t)$ with a very small interval τ (25 μs) and generate a binary spike stream S :

$$S_{i,j,k} = \begin{cases} 1, & \text{if } \exists t \in ((k-1)\tau, k\tau], V_{i,j}(t) = 0, \\ 0, & \text{otherwise,} \end{cases} \quad (2)$$

where $k = 1, 2, \dots$ is the k -th polling. Thus, for T times polling, the output S with a size of $H \times W \times T$ will be generated.

The industrial production of spike cameras has also been relatively mature, improving their availability greatly. A company called SpikeSEE, specializing in spike cameras, has been established. Their official website can be found at <https://vidar.cn/>. The commercialization of spike cameras is well-developed, making them readily available for purchase. Spike cameras have proven to be highly useful and widely applicable in research and industry. Figure 3 shows several Spike cameras that SpikeSEE has produced. According to the spatial resolution, it can be divided into two types: 100000 pixels and megapixels. The third-generation megapixel spike camera, shown at the bottom right in Figure 3, can already provide spikes for color reconstruction of high-speed scenes. Specifications for these cameras can also be found on the company's website. The industrial production of spike cameras can promote their popularization, and supporting software and algorithm development tools are also essential.

Why use spike cameras? The spike camera offers significant advantages in terms of high dynamic range and ultra-high speed due to its unique working principle. In Figure 4, we present a series of high-speed and overexposed scenes captured using the spike camera. The resulting images were obtained through the reconstruction module of SpikeCV. A comparative analysis reveals that conventional cameras suffer from motion blur and overexposure issues, whereas the spike camera excels in reducing motion blur and effectively reconstructing the texture in overexposed areas. When dealing with high-speed scenes using traditional cameras, it becomes necessary to increase the shutter speed and widen the aperture to allow sufficient light entry. However, in scenes prone to overexposure, this approach prevents the synthesis of multiple exposures through HDR imaging technology. This challenge of balancing high speed and high dynamic range is effectively addressed by leveraging the unique strengths of the spike camera. The images displayed in Figure 4 were all reconstructed using the spike camera under uniform settings, providing a clear and consistent basis for comparison.

(a) 100,000 pixel Spike Camera



(b) Megapixel Spike Camera



Figure 3 (Color online) Ultra-high-speed spike camera. (a) Chips and products of a 100000-pixel spike camera; (b) chips and products of megapixel spike cameras. After the colon of each camera is its product type.

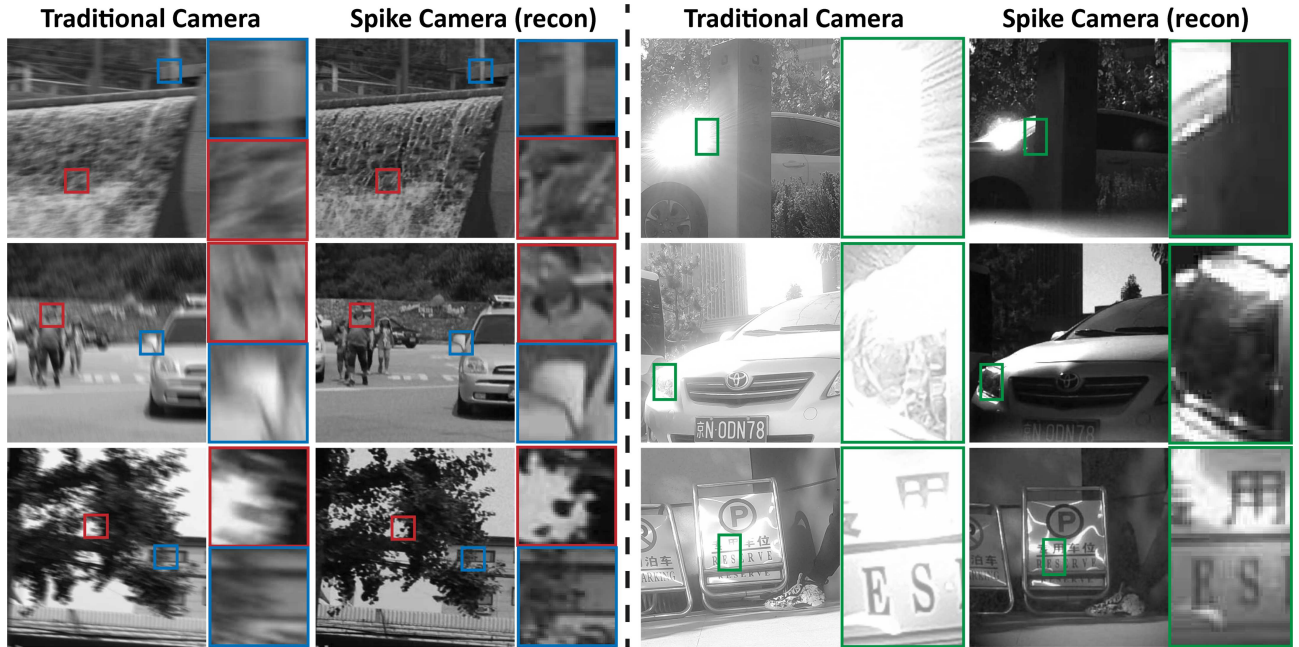


Figure 4 (Color online) The low latency and high dynamic range advantages of spike cameras. From left to right are high-speed motion and high dynamic range scenes recorded with traditional cameras and spike cameras, respectively. The right side of each picture is its partial enlargement.

2.2 Spike-based algorithms

The research studies related to spike cameras have made significant progress, which can be divided into three categories: coding and simulation of spikes, high-quality imaging based on spikes, and vision tasks based on spikes. These three parts correspond to the bottom-up research of the spike vision.

2.2.1 Coding and simulation of spikes

Spike coding is mainly to compress the dense binary spike streams through statistical characteristics, which is similar to the field of traditional video coding and decoding [30,31]. Spike coding methods include lossy [32] and lossless compression [33]. The research of spike simulation [14,15] is mainly to explore and improve the working mechanism of spike cameras and obtain simulated spike data under many extreme conditions, which is convenient for the development of spike vision research. Deep learning has been successfully applied to various vision/natural language tasks [34–36], but its training requires a large amount of training data. Acquiring large-scale datasets for ultra-high temporal resolution spike arrays is costly, and the spike simulator can generate large-scale spike data with ground truths, such as high-frequency images, optical flow, and depth. Zhao et al. [15] proposed the SpikingSIM simulator for generating spike data from images based on the sensor principle of the spike camera. Hu et al. [14] proposed SPCS to generate spike data and labels based on graphics models [37].

2.2.2 High-quality imaging based on spikes

Spike cameras preserve the high-resolution spatiotemporal information of continuous vision, making them ideal for image reconstruction [10,11] and high-dynamic-range imaging [19–21]. The current high-quality image reconstruction algorithms for spike cameras can be divided into statistics-based [9,13], deep learning-based methods [10], and spiking neural networks (SNNs)-based [38–40] methods [11,12,41]. The main principle of the statistical method is that the spike emission frequency is proportional to the light intensity, and the pixel value can be deduced according to the interval or the frequency of spikes in the sliding window. Deep learning-based reconstruction algorithms include supervised learning networks [42] using simulated data [10,43], and self-supervised reconstruction algorithms [44] based on blind spot networks [45]. In addition to high-speed and high-dynamic-range imaging, some recent research [46–48] focuses on obtaining super-resolved images from spike data. Xia et al. [49] proposed to use the spike camera to assist video frame interpolation for traditional cameras.

2.2.3 Vision tasks based on spikes

Besides reconstructing images, there are some vision algorithm methods using spatiotemporal information directly from spike data, including networks for optical flow [14,50], depth [16,17], recognition [51], detection and tracking [3,52,53]. SCFlow [14] is a deep pyramidal network with a motion-adaptive representation for spike-based optical flow, trained by graphics-based simulated data. Spike2Flow [50] explores the continuousness of spike data in optical flow. Chen et al. [54] proposed a self-supervised method for spike-based optical flow. Wang et al. [16] proposed the first stereo depth network for spike data with Transformers based on self and cross-attention. Zhang et al. [17] proposed to use a 3D Swin Transformer to represent spike stream and a spatial-temporal Transformer to estimate monocular depth. Huang et al. [3] and Zheng et al. [18] proposed to use SNNs to implement high-speed detection and tracking through spike cameras. Zhu et al. [53] used FPGA to improve the speed of spike-based detection and tracking. Li et al. [52] proposed to use both spike camera and event camera to imitate the retina for object detection. Zhang et al. [55] proposed a temporal-ordered representation for spike streams, improving the performance of spike-based image reconstruction and segmentation.

3 Design principles

3.1 Usability

3.1.1 Encapsulation for spikes

The spike camera has released its second generation, and there will be new ones with extra characteristics whose spatial resolution, sampling rate, and data channels may be various. Besides, implementations and representations of spikes vary across different algorithms and different visual tasks. Therefore, with the development of spike cameras, the community requires a unified interface to represent a new modality. In SpikeCV, we design the new class SpikeStream to encapsulate the spike data.

There are three attributes of SpikeStream in default settings: the spike data variable SpikeMatrix, the spatial resolution width, and height. As shown in Figure 5, we offer two modes to initialize an instance of SpikeStream. The offline mode is used for loading spikes files. Users only need to deliver a parameter dictionary, including the file path and the resolution information. The function ‘get_spike_matrix’ and ‘get_block_spikes’ offer flexibility to load a full-length stream or a part of it. The online mode offers an interface to obtain real-time spikes from the spike camera. The function ‘get_device_matrix’ gets the handle of the device to control the process of reading data, and

```

class SpikeStream:
    def __init__(self, offline=True, **kwargs):
        self.SpikeMatrix = None
        self.offline = offline
        # assign parameters according to the type of input stream
        if self.offline:
            # read offline data from file
            self.filename, self.height, self.width = ParseKwargs(...)
        else:
            # parameters of online spike stream from spike camera
            self.decode_width, self.height, self.width = ParseKwargs(...)

    def get_device_matrix(self, _input, _framepool, block_len=500):
        ... # get spike matrix from device (spike camera)

    def get_spikes(self, _input, _framepool):
        ... # reading Thread for online spike streams

    def get_spike_matrix(self, flipud=True):
        ... # get spike matrix from offline file

    def get_block_spikes(self, begin_idx, block_len=1, flipud=True):
        ... # get block spikes with begin index from file

    ... # Other Interfaces for Developers

```

Figure 5 (Color online) Definition of the class SpikeStream.

an instance of the caching frame pool. In this function, we apply a separate thread to parse spike streams into an array-like representation from the pool. The initialization step mainly unifies the online and offline modes of the loading process.

With the SpikeStream, fresh users do not need to bother implementing a new data-loading interface starting from scratch. Moreover, SpikeStream is compatible with the whole workflow in the SpikeCV, especially with the existing spike datasets, which brings convenience for developers to build and train a model fast. Expandability is a necessity to encapsulate the spike data. For experienced users or the developer team, the object-oriented design benefits them by designing customized derived classes of the SpikeStream, adding new attributes, functions, or representations of spikes. In this way, users can encapsulate spikes freely and flexibly for more complex demands in projects.

3.1.2 Standardization for dataset interfaces

We designed standard dataset interfaces in SpikeCV for various vision tasks for spike cameras. To support diverse tasks, we survey and collect the currently released dataset for spike cameras, whose detail is shown in Subsection 4.3. The standard interfaces are compatible with various kinds of ground truths, including those for reconstruction, optical flow, monocular depth, stereo, detection, tracking, and recognition. Besides, the dataset interfaces in SpikeCV use standard parameter passing modes and support different file structures. Users can easily define dataset interfaces for their tasks based on the label interfaces, standard parameter passing, and standard SpikeStream class for reading spikes.

3.1.3 Modularization for vision tasks

The comprehensive modularization of SpikeCV enables different vision tasks, such as image reconstruction, optical flow, object detection, and depth estimation, to be easily implemented and integrated into the framework. This modular design allows developers to easily expand and customize the framework to fit their needs. To be specific, we have designed a series of utensils for different vision tasks, such as spike pre-processing, data augmentation, data loader, visualization, and metric, which can be easily accessed and utilized by developers to improve the accuracy and assess the efficiency of different vision tasks. Additionally, the model files are separated from the main code, providing an extra level of modularization and encapsulation. SpikeCV provides a comprehensive and highly modularized solution for tackling a wide range of visual tasks, making it easy for developers to customize and improve algorithms.

3.2 Real-time

Real-time is an important principle of SpikeCV to take advantage of the low latency of the spike streams. In order to achieve the online application, we carry out real-time processing from two aspects of data and reasoning. The detail of the real-time pipeline will be described in Subsection 4.1.

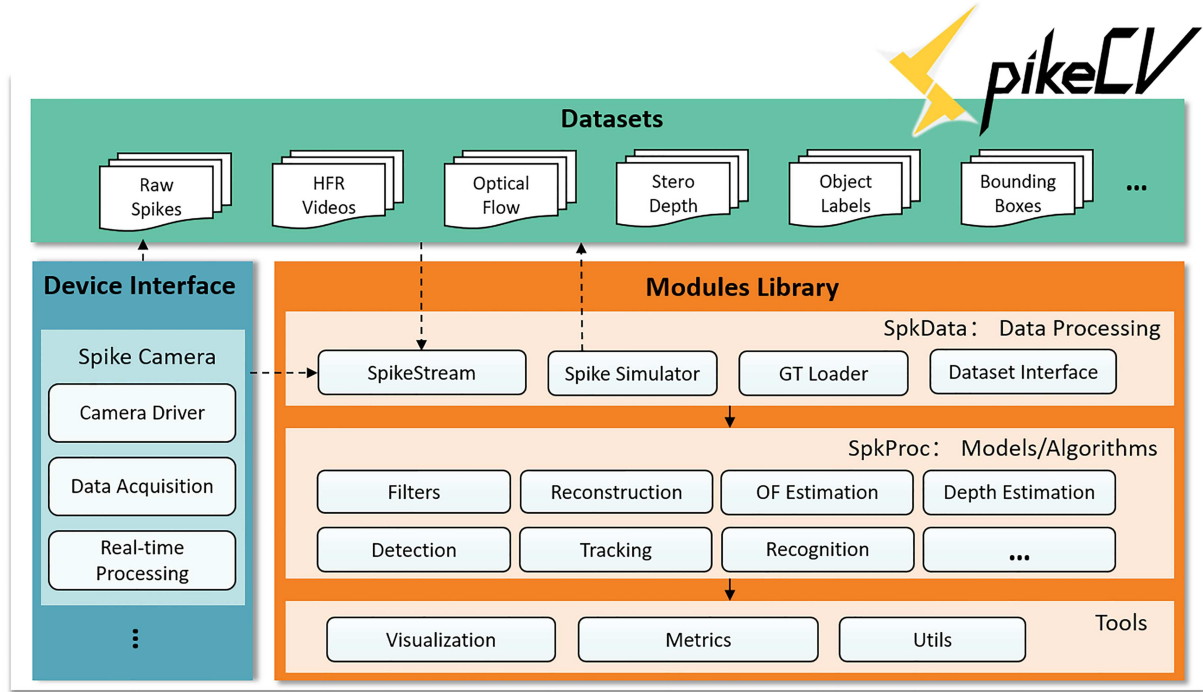


Figure 6 (Color online) Illustration of the structure of the SpikeCV.

3.2.1 Real-time data

To obtain real-time spike stream data from the spike camera, we design a multi-level thread for reading ultra-high temporal resolution spikes. Through the synchronous transmission of data streams between multiple C++ thread pools, spike data can be supplied to applications in real time without being blocked and lost.

3.2.2 Real-time inference

Coupled with real-time data loading, we aim to achieve real-time inference and prediction. The encapsulated SpikeStream instance can be produced from the pieces of real-time data from the spike camera, and supplied to multiple parallel threads of different models. Each algorithm thread realizes the real-time operation of the spike vision application by simultaneously processing the latest data.

3.3 Spike ecosystem

We aim to build up a comprehensive community ecology for the spike vision to facilitate the progress of scientific research as well as enterprise projects. For the community of image-based vision, many well-developed open-source libraries, like OpenCV [22], benefit developers with various tools for image processing. As a novel type of vision sensor, spike cameras require an open platform to release available resources, which benefits researchers in getting familiar with spike vision and getting involved in practice quickly. Different from the design principles of existing image-based open-source libraries, SpikeCV not only provides spike processing tools and spike-based visual algorithms but also provides spike camera hardware interfaces and normalized spike datasets. In this highly integrated and modularized platform, beginners can thoroughly learn what spike data is and how to use spike cameras to tackle visual tasks. It helps draw more attention to establish the community and facilitate the ecology of spike vision.

4 Details of SpikeCV

4.1 Architecture

As illustrated in Figure 6, SpikeCV consists of three parts: spike datasets, module libraries, and device interfaces.

- **Dataset.** Normalized datasets that can be used to validate or train intelligent models. The dataset files include the real scene generated by the spike camera or the spike simulator, and the corresponding label information such

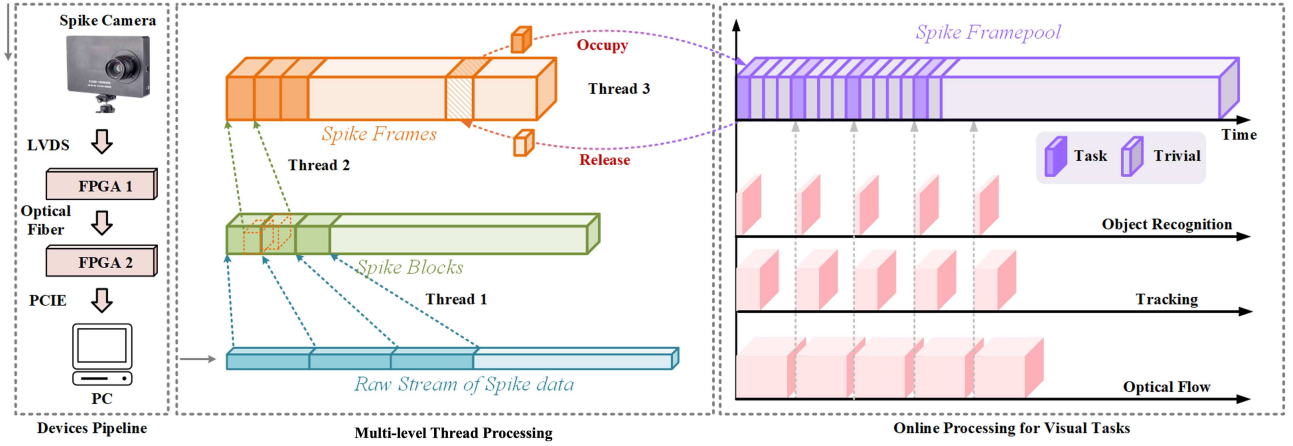


Figure 7 (Color online) Illustration of the real-time pipeline of SpikeCV. The pipeline includes the strategy of parsing spike data with multi-level threading management and real-time processing for multiple tasks.

as video, optical flow, depth, and object category. We also provide supplementary configuration files for recording properties of spike data and label files.

- **Modules library.** The module library includes three categories: data processing, vision model/algorithm, and tools. The data processing module SpkData includes the spike stream class, the spike simulator, the label loader, and the dataset interface for the training model. The algorithm library SpkProc contains various vision algorithms for spike cameras, and the Tool is a supporting development tool library responsible for visualization, metrics, and development utensils.

- **Device interface.** The device interface is to facilitate the user to apply the spike-based algorithm to the real-time processing hardware. Currently, SpikeCV has integrated the spike camera. Users can use our spike camera interface to customize the scene to collect datasets or evaluate the real-time performance of the algorithm.

Datasets and device interfaces are the two sources of data in SpikeCV for obtaining spike streams. These two sources of data have different purposes. The dataset mainly provides spike streams and corresponding labeled data that can be used for various visual tasks, which is convenient for evaluating the performance of spike-based algorithms or training machine learning models for different tasks. In addition to spike files, the dataset we provided also contains configuration files representing their spatial resolution, and label data properties. For data that has not been labeled, we use ‘raw’ to indicate its label type. In addition to the datasets we provide, users can also use spike cameras through hardware interfaces to obtain real-time spikes to evaluate the performance of their models in various scenarios or build their datasets by customizing scenarios according to task requirements. Except for using the device interface to obtain real-world spike streams, users can also use the spike simulator in SpikeCV to convert high frame rate video or picture sequences into corresponding spikes, and quickly obtain labeled spike datasets.

Real-time pipeline. We aim to synchronize the acquisition of data and the process of algorithms in a real-time scheme in the SpikeCV. As illustrated in Figure 7, we design a pipeline that can parse user-friendly spike frames from raw outputs of the hardware and process multiple visual tasks in real time. The pipeline mainly consists of two parts: multi-level threads for reading real-time spikes and online visual applications.

The first part is mainly implemented in the SpikeLinkInput Library of SpikeCV with C++. Developers can also modify the C++ code to meet different requirements. In this part, three threads are responsible for collecting spikes at different stages. Thread 1 takes the raw stream of spikes and packs them into blocks that are arranged into a queue LibBlockQueue. The blocks contain spikes and other information like frame headers. Thread 2 parses the spike blocks and assembles spikes into streams of bytes arranged in the spatial and temporal order. When the user enters a command to get camera data, these byte streams are continuously fetched and put into the queue Lib-FramePool. Thread 3 takes charge of handling spike frames between the C++ library and the user’s APP in the Python program. In this thread, the byte streams are converted to pieces of user-friendly frames with the size of $H \times W \times T_{\text{csum}}$, where $H \times W$ is the spatial resolution of the camera sensor and T_{csum} is a customized length of spike frames in one piece. The APP also maintains a queue App-FramePool, and Thread 3 always delivers the data address from Lib-FramePool to App-FramePool. After occupying the data, App-FramePool will release the address back to Lib-FramePool.

The second part is the mechanism of simultaneously processing multiple tasks cooperated with the App-FramePool in real-time. Figure 7 shows an example of three tasks processed by three independent threads, and the pink boxes denote the single-inference time cost. The main program always takes out frames from the App-FramePool. One

Table 1 Spike-based algorithm package in SpikeCV.

Function	Package name	Input	Output
Filters	spkProc.filters		Filtered spikes
Reconstruction	spkProc.reconstruction		Images/videos matrix
Optical flow estimation	spkProc.optical_flow		Optical flow
Depth estimation	spkProc.depth_estimation	SpikeStream	Stereo/monocular depth
Object detection	spkProc.detection		bbox and type
Multi-object tracking	spkProc.tracking		bbox and trajectory
Object recognition	spkProc.recognition		Object label
Data augmentation	spkProc.augment		Augmented spikes

```

from spkProc.tracking.spike_sort import SpikeSORT
from visualization.get_video import obtain_detection_video, vis_trajectory

# MOT model construction
spike_tracker = SpikeSORT(spike_h, spike_w, device)
spike_tracker.calibrate_motion(spikes, calibration_time)
spike_tracker.get_results(spikes, tracking_file)
spike_tracker.save_trajectory(trajectories_filename)

# visualization
vis_trajectory(tracking_file, trajectories_filename, **para_dict)
obtain_mot_video(spikes, video_filename, tracking_file, **para_dict)

# measure performance
metrics = TrackingMetrics(tracking_file, **paraDict)
metrics.get_results()

```

Figure 8 (Color online) Example of building a multi-object tracking model using SpikeCV.

frame will be processed when three threads are all in idle states, otherwise, its pointer will be released right back. In this way, algorithms for different tasks always process the same frame, and the App-FramePool would not be blocked.

Spike loader. Data loading and processing are the keys to building models. As described in Subsection 3.1.1, we design SpikeStream as a data type for spikes to be compatible with different vision tasks and different data sources. Users can construct continuous spike stream data through SpikeStream, or convert it into a NumPy array, or a tensor to perform matrix-based programming, and conveniently use the library algorithms supported by these data types.

For data generated by the spike camera, SpikeStream will use a dedicated read thread to obtain the high-speed spike stream generated by the spike camera running in real time. These spike streams will be buffered into a memory pool by the input callback function of the hardware interface. The reading thread in SpikeStream reads from the memory pool at high speed, forms a temporally continuous spike matrix, and returns it for online processing. SpikeStream mainly represents the data type of the spike stream. To distinguish and unify the datasets corresponding to different vision tasks, we provide a data dictionary method ‘data_parameter_dict’ that specifically represents the attributes of the dataset. Users only need to specify the task type and the name of the dataset to use the obtained data dictionary to build the SpikeStream class or build a data loader for training machine learning models.

Model construction. We provide various spike-based visual algorithms in SpikeCV, and users can choose different modules or combine them to construct the model according to task requirements. Table 1 introduces the Python package names, input, and output of the algorithm modules. These algorithms include adjusted traditional vision methods based on spikes (e.g., SVM [56]), SNNs [3, 11], and deep-learning models [17, 44, 50].

Reusability and deployability of models are important properties of the modules for SpikeCV. When creating a model, users can train the model according to task goals or directly apply our vision algorithm. Taking the multi-object tracking task as an example, the currently integrated SNN-based algorithm [3] is an online model, which does not require a large amount of data for model training. As shown in Figure 8, developers can build the model with a few lines of code and directly apply the algorithm to track objects. With the obtained results, users can use the Visualization package in SpikeCV to display the candidate boxes for detection and the trajectory of each object. If there is ground truth data for the corresponding task, the Metrics module can be applied to quantitatively measure the performance of the models.

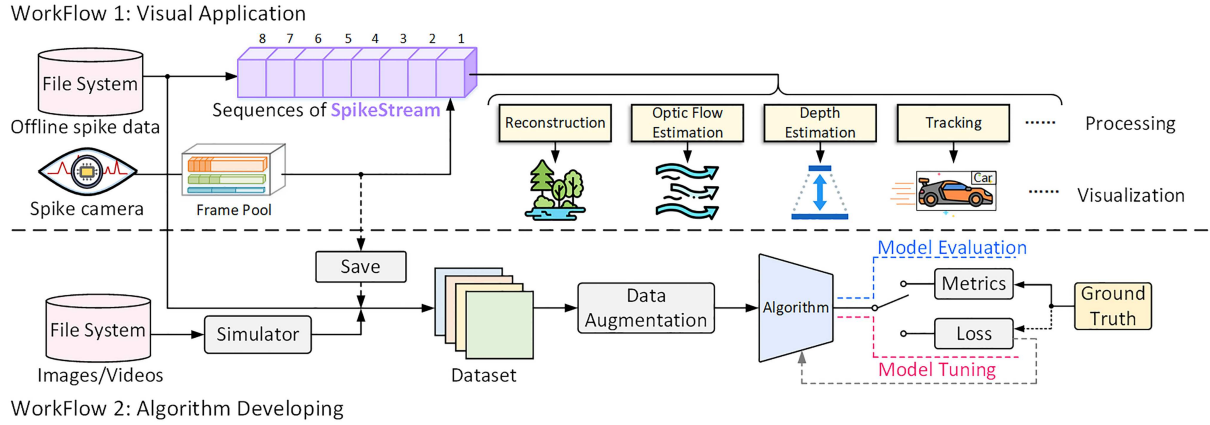


Figure 9 (Color online) Illustration of the workflow of SpikeCV.

4.2 Workflow

The workflow of SpikeCV meets the common needs of visual tasks and visualization. Figure 9 shows the workflow of SpikeCV, which contains two parts of the visual application and algorithm development. A typical SpikeCV workflow for developers is to conduct visual applications that involve data loading, processing, and visualization. Developers can utilize the data loading interface to load offline sequences of spike stream to the memory or collect real-time stream from a real-time spike camera. The real-time stream can be either processed in memory or saved to the file system for offline evaluation. The processing part manipulates the memory data and is responsible for high-quality reconstruction as well as performing various visual tasks. Developers can run vision tasks, including optical flow estimation, depth estimation, object tracking, object detection, and object recognition. After processing the spike stream, developers can visualize the task output through the provided visualization module. These tools can generate continuous images and show the visualized results of different tasks, such as a series of optical flows and depth maps.

To support developers in designing more effective spike processing algorithms, we provide a comprehensive suite of tools to streamline the development workflow. A high-quality dataset is crucial for algorithm development, and we offer three methods for obtaining datasets: using pre-defined offline datasets, simulating spike streams from images or videos via simulators, or collecting real-world data by capturing real-time streams. Once the dataset is prepared and labeled, developers can leverage encapsulated dataset modules to load custom datasets and efficiently begin model tuning and evaluation. To enhance model generalization and robustness, common data augmentation techniques such as rotation, flipping, and resizing can be applied. For model evaluation, developers can utilize pre-defined metrics and loss functions to validate their models against ground truth data.

4.3 Datasets

SpikeCV offers detailed descriptions and interfaces for most of the currently released datasets for spike cameras. The details of the datasets are shown in Table 2. According to the ways to obtain the data, these datasets can be divided into real-world and synthetic data. Some of the real-world datasets have no labels and are mainly for subjective quality assessment for various tasks. The synthetic datasets are generated with different kinds of spike simulators. All of the synthetic datasets have image ground truths, and some of the synthetic datasets have ground truths of optical flow and depth maps. SpikeCV offers dataset interfaces that are easy to use, and an example of defining a dataset interface is shown in Procedure 1.

5 Verification applications

In this section, we use the datasets, hardware interfaces, and module libraries in SpikeCV to implement offline inference tests and real-time applications for various vision tasks, respectively. All experiments are performed employing the SpikeCV module. Many sample codes are also provided in SpikeCV to help novices quickly get started with spike vision applications.

Procedure 1 Using dataset interface in SpikeCV.

```

1: /* Definition of the dataset: */
2: Initialize the parameters of the dataset;
3: Construct the path_list of the dataset;
4: Get samples based on the parameters and paths:
5:   (a) Construct data_parameter_dict;
6:   (b) Construct SpikeStream object and other data;
7:   (c) Return the gotten data;
8: /* Using of the dataset: */
9: Define parameters in data_parameter_dict;
10: Create an object of the dataset class based on the defined parameters;
11: Construct the loader using DataLoader in Pytorch.

```

Table 2 Open-source datasets about spike cameras.

	Dataset	Label	Description	Resolution
Real-captured	recVidarReal2019 [41]	–	Scenes with high-speed objects or spike camera	250×400
	momVidarReal2021	–	Various objects with different kinds of motion patterns	250×400
	PKU-Retina-Recon [13]	–	Spike and event data from the same scenes	250×400
	motVidarReal2020 [18]	Tracking	Multi-object detection dataset for spike data	250×400
	PKU-Vidar-DVS [52]	Detection	Detection dataset for joint imaging with spike and event	250×400
	PKU-Spike-Stereo [16]	Depth	Stereo depth dataset for spike data	250×400
	Outdoor-Spike [17]	–	Spike data from autonomous driving scenes	250×400
Synthetic	Spike-REDS [10]	Image	Synthesized from REDS dataset [57] for reconstruction	720×1280
	Spike-Vimeo [46]	Image	Synthesized from Vimeo-90k dataset [58] for super-resolution	256×448
	SPIFT [14]	Image, flow	Simulated dataset with stochastic scenes for optical flow	500×800
	PHM [14]	Image, flow	Simulated dataset with designed scenes for optical flow	500×800
	RSSF [50]	Image, flow	Synthesized data for optical flow in real scenes	$\sim 0.6\text{--}1.3\text{MP}$
	Spike-KITTI [16]	Image, Depth	Synthesized data from KITTI [59] for stereo depth	$\sim 370 \times 1240$
	DENSE-Spike [17]	Image, Depth	Synthesized data from DENSE dataset [60] for monocular depth	346×260

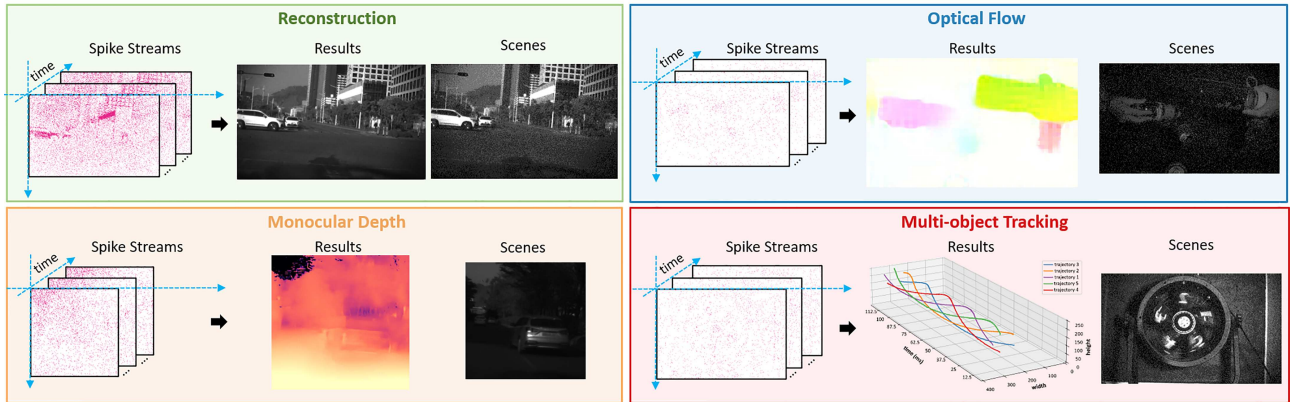


Figure 10 (Color online) Examples of vision tasks implemented based on the offline dataset PKU-Vidar-DVS in SpikeCV. Their input is the spike stream obtained by the data interface of SpikeCV. The scene in the last column of each panel is reconstructed with the TFP algorithm [13], which has been brightened for visualization.

5.1 Offline evaluation

To evaluate the reusability of each algorithm across similar scenes, we conducted offline testing using the PKU-Vidar-DVS dataset from SpikeCV. This dataset encompasses a wide range of indoor and outdoor scenes with diverse motion types, providing a rich basis for comprehensive evaluation. We test the effects of image reconstruction, optical flow estimation [50], monocular depth estimation [17], and a multi-object tracking algorithm [3] modules on this dataset. Since the PKU-Vidar-DVS dataset [52] only provides label data applicable to object detection, we subjectively evaluate the results of these four tasks by visualizing the results. The visualization results of the experiments are shown in Figure 10. It can be seen that the reproduction effects of these models in SpikeCV are promising. The left two rows of Figure 10 depict an outdoor autonomous driving scenario. In this scene, the SSML [44] reconstruction algorithm reduces noise while preserving sharp edges and contours, and the depth estimation module generates detailed depth maps with well-defined contours. The right two rows of Figure 10 show

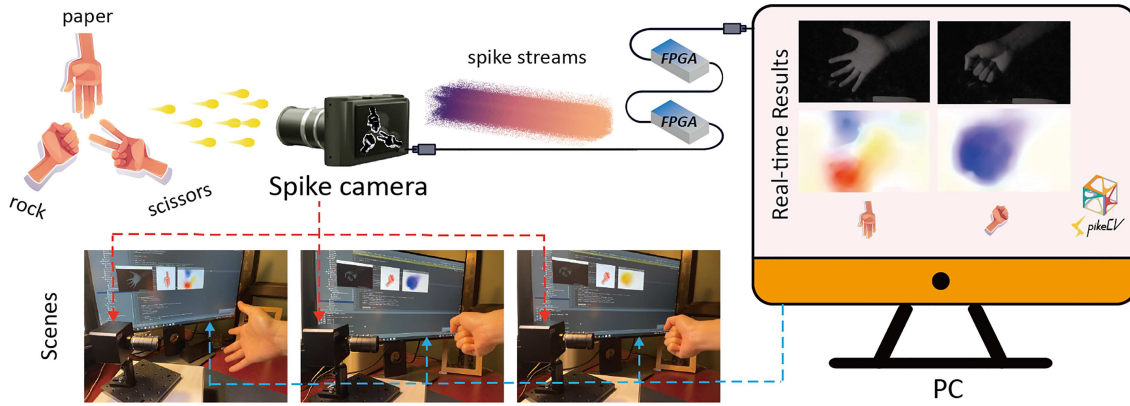


Figure 11 (Color online) Real-time application scenarios of SpikeCV in rock-paper-scissors tasks.

indoor scenes of multiple waving bottles and fast-rotating objects. The results demonstrate that, even with sparse spike streams, the algorithm module robustly estimates the optical flow and trajectory of moving objects.

5.2 Real-time applications

In the game of rock-paper-scissors, only fast no broken. The game would be easy if you saw the opponent's gesture within a very small time interval. In the real-time application, we take advantage of the spike camera to collect continuous visual information and simultaneously apply the reconstruction, optical flow estimation, and recognition algorithms in SpikeCV to process the real-time spike streams generated by the camera, followed by using the visualization module to display the results interactively. Figure 11 shows the running scenarios and results of the real-time application. As shown in the results, even with rapid changes in gestures (from paper to rock), the model constructed by SpikeCV can give high-quality results in time.

6 Conclusion and future work

In this paper, we introduce SpikeCV, an open-source platform for ultra-fast spike cameras, from aspects of design principles, workflow, and usage methods of each module in detail. At present, SpikeCV includes a rich set of normalized spike datasets, easy-to-use hardware interfaces, and a large library of spike-based algorithm modules. In the experiments, we demonstrate using SpikeCV to use offline datasets and online cameras as data sources and use each module in the algorithm library to achieve image reconstruction, optical flow estimation, depth estimation, detection, tracking, and recognition. Experimental results show the effectiveness and usability of SpikeCV.

SpikeCV also supports the joint use of spike and event-based cameras, taking advantage of their complementary strengths in applications such as HDR reconstruction and high-speed object detection. To facilitate this, we provide dual-modality datasets like the PKU-Vidar-DVS dataset for paired spike-event object detection [52] and the hybrid dataset for HDR imaging [19,20], offering a robust foundation for neuromorphic vision research in dynamic environments.

In the future, we will be committed to building a spike vision ecosystem. We will provide large-scale spike datasets that can be applied to various algorithms, and continue to supplement and improve spike-based algorithms and processing tools. For the hardware interfaces, we plan to add more interfaces of cameras to facilitate the fusion of other data with the spike camera. To take advantage of the spike camera as a neuromorphic sensor, we will provide the interface of the neuromorphic computing chip [61–64], to take advantage of low power consumption and ultra-high speed of brain-like vision algorithm [39,65–67], and apply the spike vision to various ultra-fast scene applications, such as UAV [68] and automatic driving [69].

Acknowledgements This work was supported in part by Young Elite Scientists Sponsorship Program by CAST (Grant No. 2023QNRC001), Beijing Natural Science Foundation (Grant No. 4252039), National Natural Science Foundation of China (Grant No. 62306015), and China Postdoctoral Science Foundation (Grant No. 6230070113).

References

- 1 von Neumann J, Kurzweil R. The Computer and the Brain. New Haven: Yale University Press, 2012
- 2 Dong S, Huang T, Tian Y. Spike camera and its coding methods. In: Proceedings of DCC, 2017. 437–437
- 3 Huang T, Zheng Y, Yu Z, et al. 1000× faster camera and machine vision with ordinary devices. *Engineering*, 2023, 25: 110–119
- 4 Uchihashi T, Kodera N, Ando T. Guide to video recording of structure dynamics and dynamic processes of proteins by high-speed atomic force microscopy. *Nat Protoc*, 2012, 7: 1193–1206

- 5 Haber R E, Jiménez J E, Peres C R, et al. An investigation of tool-wear monitoring in a high-speed machining process. *Sens Actuat A-Phys*, 2004, 116: 539–545
- 6 Zhang B, Zheng Y, Chen S, et al. Spikemm: flexi-magnification of high-speed micro-motions. *ArXiv:2406.00383*
- 7 Falanga D, Kleber K, Scaramuzza D. Dynamic obstacle avoidance for quadrotors with event cameras. *Sci Robot*, 2020, 5: eaaz9712
- 8 Gehrig D, Scaramuzza D. Low-latency automotive vision with event cameras. *Nature*, 2024, 629: 1034–1040
- 9 Zhao J, Xiong R, Huang T. High-speed motion scene reconstruction for spike camera via motion aligned filtering. In: *Proceedings of ISCAS*, 2020. 1–5
- 10 Zhao J, Xiong R, Liu H, et al. Spk2imgnet: learning to reconstruct dynamic scene from continuous spike stream. In: *Proceedings of CVPR*, 2021. 11996–12005
- 11 Zheng Y, Zheng L, Yu Z, et al. High-speed image reconstruction through short-term plasticity for spiking cameras. In: *Proceedings of CVPR*, 2021. 6358–6367
- 12 Zheng Y, Zheng L, Yu Z, et al. Capture the moment: high-speed imaging with spiking cameras through short-term plasticity. *IEEE Trans Pattern Anal Mach Intell*, 2023, 45: 8127–8142
- 13 Zhu L, Dong S, Huang T, et al. A retina-inspired sampling method for visual texture reconstruction. In: *Proceedings of ICME*, 2019. 1432–1437
- 14 Hu L, Zhao R, Ding Z, et al. Optical flow estimation for spiking camera. In: *Proceedings of CVPR*, 2022. 17844–17853
- 15 Zhao J, Zhang S, Ma L, et al. Spikingsim: a bio-inspired spiking simulator. In: *Proceedings of ISCAS*, 2022. 3003–3007
- 16 Wang Y, Li J, Zhu L, et al. Learning stereo depth estimation with bio-inspired spike cameras. In: *Proceedings of ICME*, 2022. 1–6
- 17 Zhang J, Tang L, Yu Z, et al. Spike transformer: monocular depth estimation for spiking camera. In: *Proceedings of European Conference on Computer Vision*, 2022. 34–52
- 18 Zheng Y, Yu Z, Wang S, et al. Spike-based motion estimation for object tracking through bio-inspired unsupervised learning. *IEEE Trans Image Process*, 2023, 32: 335–349
- 19 Han J, Yang Y, Duan P, et al. Hybrid high dynamic range imaging fusing neuromorphic and conventional images. *IEEE Trans Pattern Anal Mach Intell*, 2023, 45: 8553–8565
- 20 Han J, Zhou C, Duan P, et al. Neuromorphic camera guided high dynamic range imaging. In: *Proceedings of CVPR*, 2020. 1730–1739
- 21 Zhou C, Zhao H, Han J, et al. Unmodnet: learning to unwrap a modulo image for high dynamic range imaging. In: *Proceedings of NeurIPS*, 2020. 1559–1570
- 22 Bradski G, Kaehler A. *Learning OpenCV: Computer Vision with the OpenCV Library*. Sebastopol: O'Reilly Media, Inc., 2008
- 23 Choquette J, Gandhi W, Nvidia a100 GPU: performance & innovation for GPU computing. In: *Proceedings of IEEE Hot Chips 32 Symposium (HCS)*, 2020. 1–43
- 24 Toropov E, Buitrago P A, Moura J M. Shuffler: a large scale data management tool for machine learning in computer vision. In: *Proceedings of the Practice and Experience in Advanced Research Computing on Rise of the Machines (learning)*, 2019. 1–8
- 25 Wilson R, Lu H, Zhu M, et al. Refics: a step towards linking vision with hardware assurance. In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2022. 4031–4040
- 26 Riba E, Mishkin D, Ponsa D, et al. Kornia: an open source differentiable computer vision library for PyTorch. In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2020. 3674–3683
- 27 Oliphant T E. *Guide to NumPy*. Vol. 1. Cambridge: MIT, 2006
- 28 Virtanen P, Gommers R, Oliphant T E, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nat Methods*, 2020, 17: 261–272
- 29 Paszke A, Gross S, Massa F, et al. Pytorch: an imperative style, high-performance deep learning library. In: *Proceedings of NIPS*, 2019
- 30 Girod B, Aaron A M, Rane S, et al. Distributed video coding. *Proc IEEE*, 2005, 93: 71–83
- 31 Ohm J R, Sullivan G J, Schwarz H, et al. Comparison of the coding efficiency of video coding standards-including high efficiency video coding (HEVC). *IEEE Trans Circ Syst Video Technol*, 2012, 22: 1669–1684
- 32 Zhu L, Dong S, Huang T, et al. Hybrid coding of spatiotemporal spike data for a bio-inspired camera. *IEEE Trans Circ Syst Video Technol*, 2021, 31: 2837–2851
- 33 Dong S, Zhu L, Xu D, et al. An efficient coding method for spike camera using inter-spike intervals. In: *Proceedings of DCC*, 2019. 568–568
- 34 LeCun Y, Bengio Y, Hinton G. Deep learning. *Nature*, 2015, 521: 436–444
- 35 Ramachandram D, Taylor G W. Deep multimodal learning: a survey on recent advances and trends. *IEEE Signal Process Mag*, 2017, 34: 96–108
- 36 Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need. *ArXiv:1706.03762*
- 37 Blender Online Community. *Blender—A 3D Modelling and Rendering Package*. Amsterdam: Blender Foundation, Blender Institute, 2021
- 38 Bi G, Poo M. Synaptic modifications in cultured hippocampal neurons: dependence on spike timing, synaptic strength, and postsynaptic cell type. *J Neurosci*, 1998, 18: 10464–10472
- 39 Maass W. Networks of spiking neurons: the third generation of neural network models. *Neural Netws*, 1997, 10: 1659–1671
- 40 Tsodyks M V, Markram H. The neural code between neocortical pyramidal neurons depends on neurotransmitter release probability. *Proc Natl Acad Sci USA*, 1997, 94: 719–723
- 41 Zhu L, Dong S, Li J, et al. Retina-like visual image reconstruction via spiking neural model. In: *Proceedings of CVPR*, 2020. 1438–1446
- 42 Wu X, Barthel E, Zhang W. Piecewise 2D autoregression for predictive image coding. In: *Proceedings of ICIP*, 1998. 901–904
- 43 Zhao J, Xiong R, Xie J, et al. Reconstructing clear image for high-speed motion scene with a retina-inspired spike camera. *IEEE Trans Comput Imag*, 2022, 8: 12–27
- 44 Chen S, Duan C, Yu Z, et al. Self-supervised mutual learning for dynamic scene reconstruction of spiking camera. In: *Proceedings of IJCAI*, 2022. 2859–2866
- 45 Lehtinen J, Munkberg J, Hasselgren J, et al. Noise2noise: learning image restoration without clean data. In: *Proceedings of ICML*, 2018. 2965–2974
- 46 Xiang X, Zhu L, Li J, et al. Learning super-resolution reconstruction for high temporal resolution spike stream. *IEEE Trans Circ Syst Video Technol*, 2023, 33: 16–29
- 47 Zhao J, Xie J, Xiong R, et al. Super resolve dynamic scene from continuous spike streams. In: *Proceedings of ICCV*, 2021. 2533–2542
- 48 Zhao J, Xiong R, Zhang J, et al. Learning to super-resolve dynamic scenes for neuromorphic spike camera. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023. 3579–3587
- 49 Xia L, Zhao J, Xiong R, et al. Svfi: spiking-based video frame interpolation for high-speed motion. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023. 2910–2918
- 50 Zhao R, Xiong R, Zhao J, et al. Learning optical flow from continuous spike streams. In: *Proceedings of NIPS*, 2022. 7905–7920
- 51 Zhao J, Zhang S, Yu Z, et al. SpiReco: fast and efficient recognition of high-speed moving objects with spike camera. *IEEE Trans Circ Syst Video Technol*, 2024, 34: 5856–5867
- 52 Li J, Wang X, Zhu L, et al. Retinomorphic object detection in asynchronous visual streams. In: *Proceedings of AAAI*, 2022. 1332–1340
- 53 Zhu Y, Zhang Y, Xie X, et al. An FPGA accelerator for high-speed moving objects detection and tracking with a spike camera. *Neural Comput*, 2022, 34: 1812–1839
- 54 Chen S, Yu Z, Huang T. Self-supervised joint dynamic scene reconstruction and optical flow estimation for spiking camera. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023. 350–358

- 55 Zhang J, Jia S, Yu Z, et al. Learning temporal-ordered representation for spike streams based on discrete wavelet transforms. In: Proceedings of the AAAI Conference on Artificial Intelligence, 2023. 137–147
- 56 Hearst M A, Dumais S T, Osuna E, et al. Support vector machines. *IEEE Intell Syst Their Appl*, 1998, 13: 18–28
- 57 Nah S, Baik S, Hong S, et al. Ntire 2019 challenge on video deblurring and super-resolution: dataset and study. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, 2019
- 58 Xue T, Chen B, Wu J, et al. Video enhancement with task-oriented flow. *Int J Comput Vis*, 2019, 127: 1106–1125
- 59 Menze M, Geiger A. Object scene flow for autonomous vehicles. In: Proceedings of CVPR, 2015. 3061–3070
- 60 Hidalgo-Carri6 J, Gehrig D, Scaramuzza D. Learning monocular dense depth from events. In: Proceedings of 3DV, 2020. 534–542
- 61 Akopyan F, Sawada J, Cassidy A, et al. TrueNorth: design and tool flow of a 65 mW 1 million neuron programmable neurosynaptic chip. *IEEE Trans Comput-Aided Des Integr Circ Syst*, 2015, 34: 1537–1557
- 62 Davies M, Srinivasa N, Lin T H, et al. Loihi: a neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 2018, 38: 82–99
- 63 Furber S B. Brain-inspired computing. *IET Comput Dig Tech*, 2016, 10: 299–305
- 64 Pei J, Deng L, Song S, et al. Towards artificial general intelligence with hybrid Tianjic chip architecture. *Nature*, 2019, 572: 106–111
- 65 Neftci E O, Mostafa H, Zenke F. Surrogate gradient learning in spiking neural networks: bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Process Mag*, 2019, 36: 51–63
- 66 Tan W, Patel D, Kozma R. Strategy and benchmark for converting deep Q-networks to event-driven spiking neural networks. In: Proceedings of AAAI, 2021. 9816–9824
- 67 Wunderlich T C, Pehle C. Event-based backpropagation can compute exact gradients for spiking neural networks. *Sci Rep*, 2021, 11: 12829
- 68 Zhang P, Zhong Y, Li X. Slimyolov3: narrower, faster and better for real-time UAV applications. In: Proceedings of CVPRW, 2019
- 69 Muller U, Ben J, Cosatto E, et al. Off-road obstacle avoidance through end-to-end learning. In: Proceedings of NIPS, 2005